

IoT-Based Temperature and Humidity Monitoring System Using ESP32 and DHT22

This project is an **IoT-based temperature and humidity monitoring system** developed using an **ESP32 microcontroller** and a **DHT22 temperature and humidity sensor**. The measured environmental data is displayed in the serial monitor and uploaded to **ThingSpeak** through Wi-Fi for remote monitoring and data visualization.

1. Problem Statement

Monitoring temperature and humidity manually can be inconvenient, especially when continuous environmental data is required.

The objective of this project is to develop a simple IoT system that can:

- Measure temperature and humidity continuously.
- Process the sensor readings using an ESP32.
- Connect to the Internet through Wi-Fi.
- Upload the collected data to the ThingSpeak IoT platform.
- Allow the collected data to be monitored remotely.

2. Objectives

The main objectives of this project are:

- To measure **temperature** using a DHT22 sensor.
- To measure **relative humidity** using the DHT22 sensor.
- To interface the DHT22 sensor with an ESP32.
- To connect the ESP32 to a Wi-Fi network.
- To send sensor readings to ThingSpeak using an HTTP request.
- To display temperature and humidity readings through the serial monitor.
- To enable remote monitoring of environmental conditions using ThingSpeak.
- To demonstrate the working of a basic IoT data-logging system.

3. Components and Software Used

Hardware Components

Component	Quantity	Purpose
ESP32 Development Board	1	Main microcontroller and Wi-Fi connectivity
DHT22 Sensor	1	Measures temperature and humidity
Connecting Wires	As required	Electrical connections
Computer/Laptop	1	Programming and simulation

Software and Platforms

- **Wokwi** – Used to design and simulate the circuit.
- **MicroPython** – Programming language used for the ESP32.
- **ThingSpeak** – Cloud IoT platform used to store and visualize sensor data.
- **Web Browser** – Used to access Wokwi and ThingSpeak.

Python/MicroPython Libraries Used

- **network**– Used for Wi-Fi connectivity.
- **time**– Used for delays and timing.
- **dht**– Used to interface with the DHT22 sensor.
- **machine**– Used to access ESP32 hardware pins.
- **urequests**– Used to send HTTP requests to the ThingSpeak server.

4. Circuit Diagram

The circuit consists of an **ESP32** connected to a **DHT22 temperature and humidity sensor**.

Connections

DHT22 Pin	ESP32 Connection
VCC	3.3V
DATA	GPIO 15
GND	GND

The DHT22 data pin is connected to **GPIO 15** of the ESP32, as specified in the program:

```
sensor = dht.DHT22(machine.Pin(15))
```

Circuit Working

The DHT22 receives power from the ESP32. The sensor communicates temperature and humidity data to the ESP32 through its data pin. The ESP32 then processes these readings and sends them to ThingSpeak through a Wi-Fi connection.

The complete circuit is simulated in Wokwi.

Wokwi Project Link:\

<https://wokwi.com/projects/475786429442396161>

5. Working Principle

The project works according to the following sequence:

The ESP32 initializes the required

MicroPython libraries. The DHT22 sensor is

configured on **GPIO 15**.

The ESP32 activates its Wi-Fi interface.

The ESP32 connects to the configured Wi-Fi network.

The program waits until the Wi-Fi connection is

established. Once connected, the ESP32 obtains its

IP address.

The DHT22 sensor performs a

measurement. The ESP32 reads:

- Temperature in degrees Celsius.
- Relative humidity in percentage.

The measured values are printed on the serial monitor.

The ESP32 creates a ThingSpeak update URL containing the

sensor values. An HTTP GET request is sent to ThingSpeak.

ThingSpeak stores the temperature and humidity values in the configured channel.

The program waits for approximately **20 seconds** before taking and uploading the next reading. This process continues continuously

6. Program Explanation

The program is written in **MicroPython** for the ESP32.

Importing Libraries

```
import
network
import time
import dht
import
machine
import
urequests
```

These libraries provide Wi-Fi connectivity, timing functions, DHT22 sensor support, hardware-pin access, and HTTP requests.

Wi-Fi Configuration

```
WIFI_SSID = "Wokwi-GUEST"
WIFI_PASSWORD = ""
```

The ESP32 is configured to connect to the Wokwi virtual Wi-Fi network.

ThingSpeak API Key

```
THINGSPEAK_API_KEY = "YOUR_API_KEY"
```

The API key is used to authorize the ESP32 to update the ThingSpeak channel.

Note: The actual API key should not be published in a public README or shared repository. Replace it with your own key when running the program.

DHT22 Sensor Initialization

```
sensor = dht.DHT22(machine.Pin(15))
```

This initializes the DHT22 sensor and connects its data line to **GPIO 15** of the ESP32.

Connecting to Wi-Fi

```
wifi =
network.WLAN(network.STA_IF)
wifi.active(True)
wifi.connect(WIFI_SSID,
WIFI_PASSWORD)
```

The ESP32 enables station mode and attempts to connect to the specified Wi-Fi network. The program waits until the connection is established:

```
while not wifi.isconnected():  
    print(".", end="")  
    time.sleep(1)
```

After connecting, the IP address is displayed:

```
print("IP Address:", wifi.ifconfig()[0])
```

Reading Sensor Data

```
sensor.measure()  
  
temperature = sensor.temperature()  
humidity = sensor.humidity()
```

The DHT22 performs a measurement and provides:

- Temperature in °C.
- Humidity in %.

The readings are then displayed:

```
Print("Temperature:", temperature, "°C")  
print("Humidity:", humidity,
```

Sending Data to ThingSpeak

The program constructs a ThingSpeak update URL:

```
url = (  
    "http://api.thingspeak.com/update"  
    "?api_key=" +  
    THINGSPEAK_API_KEY +  
    "&field1=" + str(temperature) +  
    "&field2=" + str(humidity)  
)
```

The sensor values are assigned as follows:

- **Field 1** □ **Temperature**
- **Field 2** □ **Humidity**

The HTTP request is sent using:

```
response = urequests.get(url)
```

The ThingSpeak server response is displayed in the serial monitor.

Delay Between Measurements

```
time.sleep(20)
```

The system waits approximately **20 seconds** before taking the next measurement and uploading it.

Error Handling

The program uses a try-except block:

```
except Exception as e:  
    print("Error:", e)
```

If an error occurs during sensor reading or data transmission, the error message is printed instead of stopping the entire program.

7. Output

The ESP32 displays the sensor readings in the serial

monitor. Example output:

```
WiFi Connected!  
IP Address: <ESP32 IP Address>
```

```
Temperature: 24.0 °C  
Humidity: 40.0 %  
ThingSpeak Response: 2
```

```
Temperature: 24.0 °C  
Humidity: 40.0 %  
ThingSpeak Response: 3
```

```
Temperature: 24.0 °C  
Humidity: 40.0 %  
ThingSpeak Response: 4
```

The temperature and humidity values are continuously measured and uploaded to ThingSpeak.

The ThingSpeak response indicates that the update request has been received by the server. The response value changes as new updates are successfully submitted.

8. Applications

This IoT-based temperature and humidity monitoring system can be used in several applications, such as:

Home environment monitoring

Room temperature monitoring

Greenhouse monitoring

**Agricultural environmental monitoring Storage
room monitoring**

**Server-room environmental monitoring Weather
and climate monitoring projects**

IoT and embedded-systems educational projects

Laboratory environment monitoring

Remote temperature and humidity data logging

9. Limitations

The project has the following limitations:

The DHT22 sensor has a limited measurement range and accuracy compared with more advanced sensors. The system requires a working Wi-Fi connection to upload data to ThingSpeak.

Internet connectivity problems can prevent data from being uploaded. The system depends on the availability of the ThingSpeak service.

The Wokwi simulation does not completely represent all real-world electrical and environmental conditions

The project currently monitors only temperature and humidity.

The system does not include local data storage when the Internet connection is unavailable. The API key must be kept secure and should not be publicly exposed.

Sensor readings may have small variations because of sensor accuracy and environmental conditions

10. Future Scope

The project can be further improved by adding:

OLED/LCD display for displaying temperature and humidity locally.

Mobile notifications when temperature or humidity exceeds a specified limit. Additional sensors such as air-quality, light, pressure, or soil-moisture sensors. Automatic fan or cooling-system control based on temperature.

Automatic irrigation control for agricultural applications.

Data logging to local memory or an SD card. Battery or solar power for remote deployment. Improved IoT dashboard for real-time monitoring.

Sensor calibration for improved measurement accuracy.

Offline data buffering, allowing readings to be stored and uploaded when the Internet connection becomes available again.

11. Team Member's Details

Sl. No.	Name	USN/Roll No.
1	Lekhana C R	[U03ZW24S0083]
2	Misba F	[U03ZW24S0093]
3	G V Chethana	[U03ZW24S0101]
4	N Jamuna	[U03ZW24S0098]

12. Wokwi Project Link

The project was designed and simulated using Wokwi.

Wokwi Project:\

<https://wokwi.com/projects/475786429442396161>

13. ThingSpeak Channel Link

The sensor data is uploaded to ThingSpeak using the ThingSpeak API. The channel contains:

- Field 1 – Temperature (°C)
- Field 2 – Humidity (%)

ThingSpeak Channel:/ https://thingspeak.mathworks.com/channels/3466663/private_show

Project Summary

This project demonstrates a basic **IoT environmental monitoring system using ESP32 and DHT22**. The DHT22 measures temperature and humidity, while the ESP32 processes the sensor readings and connects to Wi-Fi. The readings are displayed on the serial monitor and transmitted to ThingSpeak using an HTTP GET request. ThingSpeak provides cloud-based storage and visualization of the collected data, making the system suitable as a simple platform for remote environmental monitoring.